Sorting Algorithm Performance Analysis

Team Name	Team Awesome Sauce
Members	Ben Paul, Nelson Long, Noah Kolling
Course	CS 2430
Semester	Spring 2026

Introduction

The goal of this project was to compare four sorting algorithms: Merge Sort, Quick Sort, Shaker Sort, and Heap Sort. Where we will then compare those algorithms by counting the comparisons each takes. For each sorting method, we will use some established array sizes and ensure each have an appropriate testing pool to determine actual efficiency and find worst case and best case scenarios . We used comparison count instead of runtime because runtime varies by machine.

Algorithm Summaries

Merge Sort splits the array in half recursively until each piece is one element, then merges those pieces back together in sorted order. Every merge step compares elements from two sorted sub arrays one at a time. Because the split is always symmetric, the algorithm does roughly the same amount of work no matter what the input looks like.

Quick Sort picks a pivot and rearranges the array so everything smaller than the pivot is on the left and everything larger is on the right. It then recurses on both sides. If the pivot ends up near the middle every time is is efficient.

Shaker Sort works by making repeated passes through the array, comparing each pair of adjacent elements and swapping them if they're in the wrong order. Each forward pass pushes the largest unsorted element to its correct position at the right end, and each backward pass does the same for the smallest unsorted element on the left end. After every forward-backward cycle, the search range shrinks by one on each side since those boundary elements are already in place. If a full pass completes without a single swap, the algorithm stops.

Heap Sort first reorganizes the array into a max-heap (a structure where every parent is larger than its children), then repeatedly pulls the largest element off the top and places it at the end. After each extraction, it fixes the heap. It never short-circuits based on input order, so it always does its full amount of work regardless of what the input looks like.

Methods

Generating permutations: Starting from the sorted array [0, 1, ..., n-1], it repeatedly finds the next arrangement in dictionary order until all permutations have been generated. This guarantees every possible input is tested and we can avoid duplicates

Counting comparisons: Each algorithm increments a counter exactly once per element-to-element comparison. We only count value comparisons that actually determine sorting order. Each algorithm returns the final count as an integer.

Fairness: Every algorithm received a fresh copy of each permutation before sorting, so no algorithm ever operated on a pre-partially-sorted array, that the other algorithms did not also receive.

Results

Tables 1–12 show the best 10 and worst 10 permutations for each algorithm and n value, along with the average across all permutations. Tables 13–15 summarize comparisons across all algorithms.

Table 1: Merge Sort, n = 4 (24 permutations)

Rank	Comparisons	Input Permutation
Best 1	4	[0, 1, 2, 3]
Best 2	4	[0, 1, 3, 2]
Best 3	4	[1, 0, 2, 3]
Best 4	4	[1, 0, 3, 2]
Best 5	4	[2, 3, 0, 1]

Rank	Comparisons	Input Permutation
Best 6	4	[2, 3, 1, 0]
Best 7	4	[3, 2, 0, 1]
Best 8	4	[3, 2, 1, 0]
Best 9	5	[0, 2, 1, 3]
Best 10	5	[0, 2, 3, 1]
Average	**4.67**	------
Worst 1	5	[3, 1, 2, 0]
Worst 2	5	[3, 1, 0, 2]
Worst 3	5	[3, 0, 2, 1]
Worst 4	5	[3, 0, 1, 2]
Worst 5	5	[2, 1, 3, 0]
Worst 6	5	[2, 1, 0, 3]
Worst 7	5	[2, 0, 3, 1]
Worst 8	5	[2, 0, 1, 3]
Worst 9	5	[1, 3, 2, 0]
Worst 10	5	[1, 3, 0, 2]

Table 2: Merge Sort, n = 6 (720 permutations)

Rank	Comparisons	Input Permutation
Best 1	7	[0, 1, 2, 3, 4, 5]
Best 2	7	[0, 1, 2, 3, 5, 4]
Best 3	7	[0, 2, 1, 3, 4, 5]
Best 4	7	[0, 2, 1, 3, 5, 4]

Rank	Comparisons	Input Permutation
Best 5	7	[3, 4, 5, 0, 1, 2]
Best 6	7	[3, 4, 5, 0, 2, 1]
Best 7	7	[3, 5, 4, 0, 1, 2]
Best 8	7	[3, 5, 4, 0, 2, 1]
Best 9	8	[0, 1, 2, 4, 3, 5]
Best 10	8	[0, 1, 2, 4, 5, 3]
Average	**9.83**	------
Worst 1	11	[5, 3, 2, 4, 1, 0]
Worst 2	11	[5, 3, 2, 4, 0, 1]
Worst 3	11	[5, 3, 2, 1, 4, 0]
Worst 4	11	[5, 3, 2, 1, 0, 4]
Worst 5	11	[5, 3, 1, 4, 2, 0]
Worst 6	11	[5, 3, 1, 4, 0, 2]
Worst 7	11	[5, 3, 1, 2, 4, 0]
Worst 8	11	[5, 3, 1, 2, 0, 4]
Worst 9	11	[5, 3, 0, 4, 2, 1]
Worst 10	11	[5, 3, 0, 4, 1, 2]

Table 3: Merge Sort, n = 8 (40,320 permutations)

Rank	Comparisons	Input Permutation
Best 1	12	[0, 1, 2, 3, 4, 5, 6, 7]
Best 2	12	[0, 1, 2, 3, 4, 5, 7, 6]
Best 3	12	[0, 1, 2, 3, 5, 4, 6, 7]

Rank	Comparisons	Input Permutation
Best 4	12	[0, 1, 2, 3, 5, 4, 7, 6]
Best 5	12	[0, 1, 2, 3, 6, 7, 4, 5]
Best 6	12	[0, 1, 2, 3, 6, 7, 5, 4]
Best 7	12	[0, 1, 2, 3, 7, 6, 4, 5]
Best 8	12	[0, 1, 2, 3, 7, 6, 5, 4]
Best 9	12	[0, 1, 3, 2, 4, 5, 6, 7]
Best 10	12	[0, 1, 3, 2, 4, 5, 7, 6]
Average	**15.73**	------
Worst 1	17	[7, 4, 5, 3, 6, 1, 2, 0]
Worst 2	17	[7, 4, 5, 3, 6, 1, 0, 2]
Worst 3	17	[7, 4, 5, 3, 6, 0, 2, 1]
Worst 4	17	[7, 4, 5, 3, 6, 0, 1, 2]
Worst 5	17	[7, 4, 5, 3, 2, 1, 6, 0]
Worst 6	17	[7, 4, 5, 3, 2, 1, 0, 6]
Worst 7	17	[7, 4, 5, 3, 2, 0, 6, 1]
Worst 8	17	[7, 4, 5, 3, 2, 0, 1, 6]
Worst 9	17	[7, 4, 5, 3, 1, 6, 2, 0]
Worst 10	17	[7, 4, 5, 3, 1, 6, 0, 2]

Table 4: Quick Sort, n = 4 (24 permutations)

Rank	Comparisons	Input Permutation
Best 1	4	[0, 1, 3, 2]
Best 2	4	[0, 2, 3, 1]

Rank	Comparisons	Input Permutation
Best 3	4	[0, 3, 1, 2]
Best 4	4	[0, 3, 2, 1]
Best 5	4	[1, 0, 3, 2]
Best 6	4	[1, 3, 0, 2]
Best 7	4	[2, 0, 3, 1]
Best 8	4	[2, 3, 0, 1]
Best 9	4	[3, 0, 1, 2]
Best 10	4	[3, 0, 2, 1]
Average	**4.83**	------
Worst 1	6	[0, 1, 2, 3]
Worst 2	6	[1, 0, 2, 3]
Worst 3	6	[1, 2, 0, 3]
Worst 4	6	[1, 2, 3, 0]
Worst 5	6	[1, 3, 2, 0]
Worst 6	6	[2, 1, 0, 3]
Worst 7	6	[3, 1, 2, 0]
Worst 8	6	[3, 2, 1, 0]
Worst 9	5	[2, 1, 3, 0]
Worst 10	5	[2, 3, 1, 0]

Table 5: Quick Sort, n = 6 (720 permutations)

Rank	Comparisons	Input Permutation
Best 1	8	[0, 1, 4, 3, 5, 2]

Rank	**Comparisons**	**Input Permutation**
Best 2	8	[0, 1, 4, 5, 3, 2]
Best 3	8	[0, 2, 1, 4, 5, 3]
Best 4	8	[0, 2, 1, 5, 4, 3]
Best 5	8	[0, 2, 4, 1, 5, 3]
Best 6	8	[0, 2, 4, 5, 1, 3]
Best 7	8	[0, 2, 5, 1, 4, 3]
Best 8	8	[0, 2, 5, 4, 1, 3]
Best 9	8	[0, 3, 4, 1, 5, 2]
Best 10	8	[0, 3, 4, 5, 1, 2]
Average	**10.30**	-------
Worst 1	15	[5, 4, 3, 2, 1, 0]
Worst 2	15	[5, 4, 2, 3, 1, 0]
Worst 3	15	[5, 3, 2, 1, 4, 0]
Worst 4	15	[5, 2, 4, 3, 1, 0]
Worst 5	15	[5, 2, 3, 4, 1, 0]
Worst 6	15	[5, 2, 3, 1, 4, 0]
Worst 7	15	[5, 2, 1, 3, 4, 0]
Worst 8	15	[5, 1, 2, 3, 4, 0]
Worst 9	15	[4, 3, 2, 1, 0, 5]
Worst 10	15	[4, 2, 3, 1, 0, 5]

Table 6: Quick Sort, n = 8 (40,320 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	13	[0, 1, 3, 2, 6, 5, 7, 4]
Best 2	13	[0, 1, 3, 2, 6, 7, 5, 4]
Best 3	13	[0, 1, 3, 5, 6, 2, 7, 4]
Best 4	13	[0, 1, 3, 5, 6, 7, 2, 4]
Best 5	13	[0, 1, 3, 6, 2, 5, 7, 4]
Best 6	13	[0, 1, 3, 6, 2, 7, 5, 4]
Best 7	13	[0, 1, 3, 7, 6, 2, 5, 4]
Best 8	13	[0, 1, 3, 7, 6, 5, 2, 4]
Best 9	13	[0, 1, 5, 3, 6, 2, 7, 4]
Best 10	13	[0, 1, 5, 3, 6, 7, 2, 4]
Average	**16.92**	-------
Worst 1	28	[7, 6, 5, 4, 3, 2, 1, 0]
Worst 2	28	[7, 6, 5, 3, 4, 2, 1, 0]
Worst 3	28	[7, 6, 4, 3, 2, 5, 1, 0]
Worst 4	28	[7, 6, 3, 5, 4, 2, 1, 0]
Worst 5	28	[7, 6, 3, 4, 5, 2, 1, 0]
Worst 6	28	[7, 6, 3, 4, 2, 5, 1, 0]
Worst 7	28	[7, 6, 3, 2, 4, 5, 1, 0]
Worst 8	28	[7, 6, 2, 3, 4, 5, 1, 0]
Worst 9	28	[7, 5, 4, 3, 2, 1, 6, 0]
Worst 10	28	[7, 5, 3, 4, 2, 1, 6, 0]

Table 7: Shaker Sort, n = 4 (24 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	3	[0, 1, 2, 3]
Best 2	5	[0, 1, 3, 2]
Best 3	5	[0, 2, 1, 3]
Best 4	5	[0, 3, 1, 2]
Best 5	5	[1, 0, 2, 3]
Best 6	5	[1, 0, 3, 2]
Best 7	5	[2, 0, 1, 3]
Best 8	5	[3, 0, 1, 2]
Best 9	6	[0, 2, 3, 1]
Best 10	6	[0, 3, 2, 1]
Average	**5.58**	-------
Worst 1	6	[3, 2, 1, 0]
Worst 2	6	[3, 2, 0, 1]
Worst 3	6	[3, 1, 2, 0]
Worst 4	6	[3, 1, 0, 2]
Worst 5	6	[3, 0, 2, 1]
Worst 6	6	[2, 3, 1, 0]
Worst 7	6	[2, 3, 0, 1]
Worst 8	6	[2, 1, 3, 0]
Worst 9	6	[2, 1, 0, 3]
Worst 10	6	[2, 0, 3, 1]

Table 8: Shaker Sort, n = 6 (720 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	5	[0, 1, 2, 3, 4, 5]
Best 2	9	[0, 1, 2, 3, 5, 4]
Best 3	9	[0, 1, 2, 4, 3, 5]
Best 4	9	[0, 1, 2, 5, 3, 4]
Best 5	9	[0, 1, 3, 2, 4, 5]
Best 6	9	[0, 1, 3, 2, 5, 4]
Best 7	9	[0, 1, 4, 2, 3, 5]
Best 8	9	[0, 1, 5, 2, 3, 4]
Best 9	9	[0, 2, 1, 3, 4, 5]
Best 10	9	[0, 2, 1, 3, 5, 4]
Average	**13.52**	-------
Worst 1	15	[5, 4, 3, 2, 1, 0]
Worst 2	15	[5, 4, 3, 2, 0, 1]
Worst 3	15	[5, 4, 3, 1, 2, 0]
Worst 4	15	[5, 4, 3, 1, 0, 2]
Worst 5	15	[5, 4, 3, 0, 2, 1]
Worst 6	15	[5, 4, 3, 0, 1, 2]
Worst 7	15	[5, 4, 2, 3, 1, 0]
Worst 8	15	[5, 4, 2, 3, 0, 1]
Worst 9	15	[5, 4, 2, 1, 3, 0]
Worst 10	15	[5, 4, 2, 1, 0, 3]

Table 9: Shaker Sort, n = 8 (40,320 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	7	[0, 1, 2, 3, 4, 5, 6, 7]
Best 2	13	[0, 1, 2, 3, 4, 5, 7, 6]
Best 3	13	[0, 1, 2, 3, 4, 6, 5, 7]
Best 4	13	[0, 1, 2, 3, 4, 7, 5, 6]
Best 5	13	[0, 1, 2, 3, 5, 4, 6, 7]
Best 6	13	[0, 1, 2, 3, 5, 4, 7, 6]
Best 7	13	[0, 1, 2, 3, 6, 4, 5, 7]
Best 8	13	[0, 1, 2, 3, 7, 4, 5, 6]
Best 9	13	[0, 1, 2, 4, 3, 5, 6, 7]
Best 10	13	[0, 1, 2, 4, 3, 5, 7, 6]
Average	**24.72**	--------
Worst 1	28	[7, 6, 5, 4, 3, 2, 1, 0]
Worst 2	28	[7, 6, 5, 4, 3, 2, 0, 1]
Worst 3	28	[7, 6, 5, 4, 3, 1, 2, 0]
Worst 4	28	[7, 6, 5, 4, 3, 1, 0, 2]
Worst 5	28	[7, 6, 5, 4, 3, 0, 2, 1]
Worst 6	28	[7, 6, 5, 4, 3, 0, 1, 2]
Worst 7	28	[7, 6, 5, 4, 2, 3, 1, 0]
Worst 8	28	[7, 6, 5, 4, 2, 3, 0, 1]
Worst 9	28	[7, 6, 5, 4, 2, 1, 3, 0]
Worst 10	28	[7, 6, 5, 4, 2, 1, 0, 3]

Table 10: Heap Sort, n = 4 (24 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	6	[0, 1, 3, 2]
Best 2	6	[0, 2, 3, 1]
Best 3	6	[1, 0, 3, 2]
Best 4	6	[1, 2, 3, 0]
Best 5	6	[2, 0, 3, 1]
Best 6	6	[2, 1, 3, 0]
Best 7	6	[3, 0, 1, 2]
Best 8	6	[3, 0, 2, 1]
Best 9	6	[3, 1, 0, 2]
Best 10	6	[3, 1, 2, 0]
Average	**6.50**	--------
Worst 1	7	[2, 3, 1, 0]
Worst 2	7	[2, 3, 0, 1]
Worst 3	7	[2, 1, 0, 3]
Worst 4	7	[2, 0, 1, 3]
Worst 5	7	[1, 3, 2, 0]
Worst 6	7	[1, 3, 0, 2]
Worst 7	7	[1, 2, 0, 3]
Worst 8	7	[1, 0, 2, 3]
Worst 9	7	[0, 3, 2, 1]
Worst 10	7	[0, 3, 1, 2]

Table 11: Heap Sort, n = 6 (720 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	12	[5, 0, 3, 1, 2, 4]
Best 2	12	[5, 0, 3, 2, 1, 4]
Best 3	12	[5, 0, 4, 1, 2, 3]
Best 4	12	[5, 0, 4, 2, 1, 3]
Best 5	12	[5, 1, 3, 0, 2, 4]
Best 6	12	[5, 1, 3, 2, 0, 4]
Best 7	12	[5, 1, 4, 0, 2, 3]
Best 8	12	[5, 1, 4, 2, 0, 3]
Best 9	12	[5, 2, 3, 0, 1, 4]
Best 10	12	[5, 2, 3, 1, 0, 4]
Average	**15.13**	-------
Worst 1	17	[4, 5, 2, 3, 1, 0]
Worst 2	17	[4, 5, 2, 3, 0, 1]
Worst 3	17	[4, 5, 2, 1, 3, 0]
Worst 4	17	[4, 5, 2, 0, 3, 1]
Worst 5	17	[4, 5, 1, 3, 2, 0]
Worst 6	17	[4, 5, 1, 3, 0, 2]
Worst 7	17	[4, 5, 1, 2, 3, 0]
Worst 8	17	[4, 5, 1, 0, 3, 2]
Worst 9	17	[4, 5, 0, 3, 2, 1]
Worst 10	17	[4, 5, 0, 3, 1, 2]

Table 12: Heap Sort, n = 8 (40,320 permutations)

Rank	**Comparisons**	**Input Permutation**
Best 1	21	[7, 0, 3, 1, 6, 4, 5, 2]
Best 2	21	[7, 0, 3, 1, 6, 5, 4, 2]
Best 3	21	[7, 0, 3, 2, 6, 4, 5, 1]
Best 4	21	[7, 0, 3, 2, 6, 5, 4, 1]
Best 5	21	[7, 0, 4, 1, 6, 3, 5, 2]
Best 6	21	[7, 0, 4, 1, 6, 5, 3, 2]
Best 7	21	[7, 0, 4, 2, 6, 3, 5, 1]
Best 8	21	[7, 0, 4, 2, 6, 5, 3, 1]
Best 9	21	[7, 0, 5, 1, 6, 3, 4, 2]
Best 10	21	[7, 0, 5, 1, 6, 4, 3, 2]
Average	**25.81**	----
Worst 1	29	[5, 6, 2, 7, 4, 1, 0, 3]
Worst 2	29	[5, 6, 2, 7, 4, 0, 1, 3]
Worst 3	29	[5, 6, 2, 7, 3, 1, 0, 4]
Worst 4	29	[5, 6, 2, 7, 3, 0, 1, 4]
Worst 5	29	[5, 6, 2, 4, 3, 1, 0, 7]
Worst 6	29	[5, 6, 2, 4, 3, 0, 1, 7]
Worst 7	29	[5, 6, 2, 3, 4, 1, 0, 7]
Worst 8	29	[5, 6, 2, 3, 4, 0, 1, 7]
Worst 9	29	[5, 6, 1, 7, 4, 2, 0, 3]
Worst 10	29	[5, 6, 1, 7, 4, 0, 2, 3]

Table 13: Average Comparisons Summary

Algorithm	**n = 4**	**n = 6**	**n = 8**
Merge Sort	4.67	9.83	15.73
Quick Sort	4.83	10.30	16.92
Shaker Sort	5.58	13.52	24.72
Heap Sort	6.50	15.13	25.81

Table 14: Best-Case Comparisons Summary

Algorithm	**n = 4**	**n = 6**	**n = 8**
Merge Sort	4	7	12
Quick Sort	4	8	13
Shaker Sort	3	5	7
Heap Sort	6	12	21

Table 15: Worst-Case Comparisons Summary

Algorithm	**n = 4**	**n = 6**	**n = 8**
Merge Sort	5	11	17
Quick Sort	6	15	28
Shaker Sort	6	15	28
Heap Sort	7	17	29

---

6. Analysis

Using our data to theorize Big-O, Big-Ω, and Big-Θ time complexities for the four algorithms we implemented is difficult because of our relatively small data set. Because of this, we made time complexity predictions by evaluating the structure of the algorithms then verified our projections with the data presented in the graphs below.

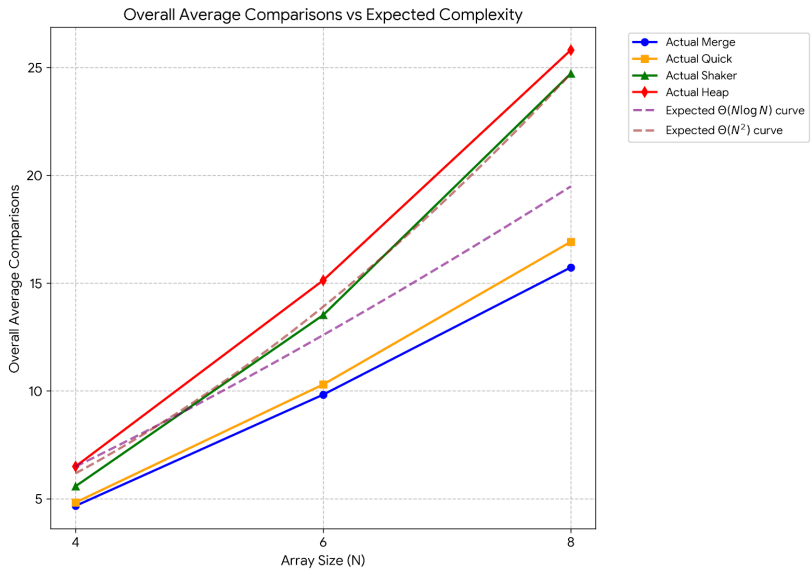
Predictions:

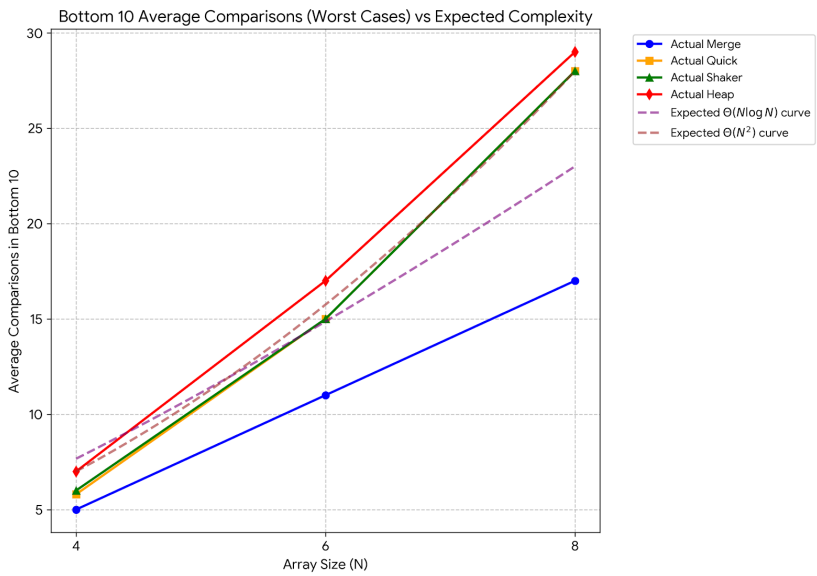
Merge Sort: Big-O: Nlog N, Big-Ω: Nlog N, Big-Θ: Nlog N
Logic: Merge sort relies on splitting the array into 1 length arrays then combining those splits together, comparing each of the elements as it does so.
Result: Looking at the graphs below, merge sort never exceeds the Nlog N curve. Therefore our limited data set supports our original prediction.

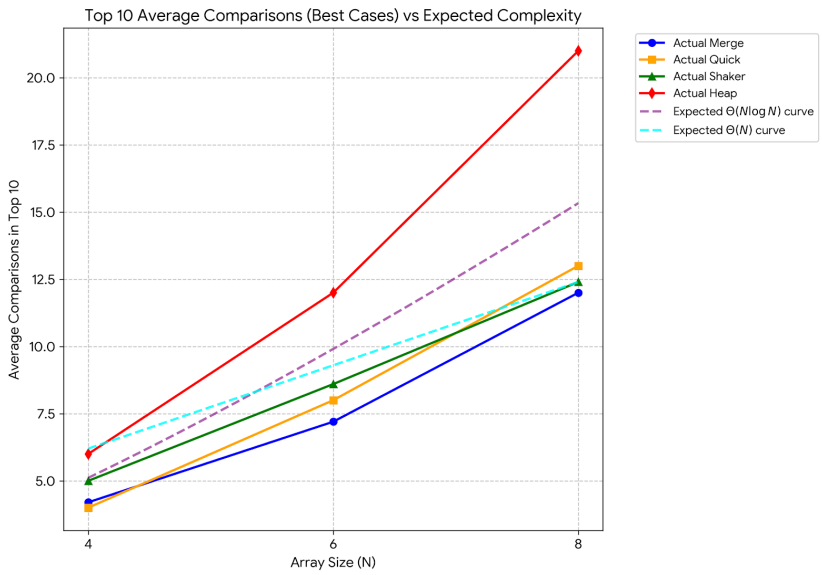
Quick Sort: Big-O: N^2, Big-Ω: Nlog N, Big-Θ: Nlog N
Logic: Since quick sort works by comparing items with a selected pivot (in our case n-1), then splitting the array and selecting new pivots, we can predict that the average and best case will be N log N. Since our pivot is always at the end of the array we can also assume an already sorted array will be the worst case and have a complexity of N^2.
Result: Based on the graphs below our quick sort results are better than N log N in its best and average cases, and is N^2 in its worst cases.

Shaker Sort: Big-O: N^2, Big-Ω: N, Big-Θ: N^2
Logic: Shaker sort compares elements adjacent to each other, sorting from left to right then right to left. Because of this, an already sorted array will only need to be passed through once which is N time complexity. In the average and worst cases it can be assumed that some items will be far from their sorted position, requiring the algorithm to pass through the array multiple times using N^2 time complexity.
Result: The graphs below show our expected results, with shaker sort aligning with N^2 complexity on average and worst cases.

Heap Sort: Big-O: Nlog N, Big-Ω: Nlog N, Big-Θ: Nlog N
Logic: Since the algorithm splits the items into a tree, giving each node two children nodes, it can be assumed that time complexity will not exceed N log N.
Result: Our graphs show that heapsort is consistently marginally above a theoretical N log N curve. While it’s not exactly on the line we predicted, we assume that as our data set grows our results will become closer to our projected complexity.


Overall Average Comparisons vs Expected Complexity
Overall Average Comparisons
Array Size (N)
Actual Merge
Actual Quick
Actual Shaker
Actual Heap
Expected $\Theta(N \log N)$ curve
Expected $\Theta(N^2)$ curve


Bottom 10 Average Comparisons (Worst Cases) vs Expected Complexity
Average Comparisons in Bottom 10
Array Size (N)
Actual Merge
Actual Quick
Actual Shaker
Actual Heap
Expected $\Theta(N \log N)$ curve
Expected $\Theta(N^2)$ curve



---

7. Conclusion

Overall, this project showed us that not all sorting algorithms are created equal. Merge Sort was the most consistent performer across all our tests and it never had a terrible input and kept its comparison counts low on average. QuickSort was close behind on average, but it fell apart on sorted inputs because of how we chose the pivot. Shaker Sort works incredibly efficiently if the array is already sorted or close to being sorted.. Heap Sort was reliable but always used more comparisons than Merge Sort, even though they are similar in many waves.

The biggest takeaway is that Big-O doesn't tell the whole story. Two algorithms can both be O(n log n) and still perform very differently in practice because of constant factors and how they handle specific input patterns.

---

8. Sources

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
3. GeeksforGeeks. "Cocktail Sort." https://www.geeksforgeeks.org/cocktail-sort/